In [1]:
```
pip install keras
```

```
Requirement already satisfied: keras in c:\users\baddd\miniconda3\lib\site-packages
(3.13.0)
Requirement already satisfied: absl-py in c:\users\baddd\miniconda3\lib\site-package
s (from keras) (2.3.1)
Requirement already satisfied: numpy in c:\users\baddd\miniconda3\lib\site-packages
(from keras) (2.2.4)
Requirement already satisfied: rich in c:\users\baddd\miniconda3\lib\site-packages
(from keras) (14.2.0)
Requirement already satisfied: namex in c:\users\baddd\miniconda3\lib\site-packages
(from keras) (0.1.0)
Requirement already satisfied: h5py in c:\users\baddd\miniconda3\lib\site-packages
(from keras) (3.15.1)
Requirement already satisfied: optree in c:\users\baddd\miniconda3\lib\site-packages
(from keras) (0.18.0)
Requirement already satisfied: ml-dtypes in c:\users\baddd\miniconda3\lib\site-packa
ges (from keras) (0.5.4)
Requirement already satisfied: packaging in c:\users\baddd\miniconda3\lib\site-packa
ges (from keras) (25.0)
Requirement already satisfied: typing-extensions>=4.6.0 in c:\users\baddd\miniconda3
\lib\site-packages (from optree->keras) (4.15.0)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\users\baddd\miniconda3\li
b\site-packages (from rich->keras) (4.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\users\baddd\miniconda3
\lib\site-packages (from rich->keras) (2.17.2)
Requirement already satisfied: mdurl~=0.1 in c:\users\baddd\miniconda3\lib\site-pack
ages (from markdown-it-py>=2.2.0->rich->keras) (0.1.2)
Note: you may need to restart the kernel to use updated packages.
```

Language Translation

In [2]:
```
import os
import shutil
import subprocess
import warnings
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.express as px
import tensorflow as tf
import tensorflow_datasets as tfds
from tensorflow import keras
from keras import layers
from colorama import Fore, Style
from IPython.core.display import HTML

warnings.filterwarnings("ignore")

K = keras.backend
ON_KAGGLE = os.getenv("KAGGLE_KERNEL_RUN_TYPE") is not None
FONT_COLOR = "#141B4D"
```

```
BACKGROUND_COLOR = "#F6F5F5"
CLR = (Style.BRIGHT + Fore.BLACK) if ON_KAGGLE else (Style.BRIGHT + Fore.WHITE)
RED = Style.BRIGHT + Fore.RED
BLUE = Style.BRIGHT + Fore.BLUE
CYAN = Style.BRIGHT + Fore.CYAN
RESET = Style.RESET_ALL
NOTEBOOK_PALETTE = {
    "DeepPlum": "#3F384A",
    "RubyRed": "#E04C5F",
    "SunburstOrange": "#FFB74D",
}


def download_dataset_from_kaggle(user, dataset, directory):
    command = "kaggle datasets download -d "
    filepath = directory / (dataset + ".zip")

    if not filepath.is_file():
        subprocess.run((command + user + "/" + dataset).split())
        filepath.parent.mkdir(parents=True, exist_ok=True)
        shutil.unpack_archive(dataset + ".zip", "data")
        shutil.move(dataset + ".zip", "data")


HTML(
    """
<style>
code {
    background: rgba(42, 53, 125, 0.10) !important;
    border-radius: 4px !important;
}
</style>
"""
)
```

```
c:\Users\BaddD\miniconda3\Lib\site-packages\keras\src\export\tf2onnx_lib.py:8: Futur
eWarning: In the future `np.object` will be defined as the corresponding NumPy scala
r.
  if not hasattr(np, "object"):
```

Out[2]:

In [24]:
```
easy_dataset_path = "dataset/eng_-french.csv"
easy_dataset = pd.read_csv(easy_dataset_path, encoding="cp1252", engine="pyarrow",
easy_dataset = easy_dataset.sample(len(easy_dataset), random_state=42)
easy_dataset.head()
```

Out[24]:

	English words/sentences	French words/sentences
2785	Take a seat.	Prends place !
29880	I wish Tom was here.	J'aimerais que Tom soit lÃ .
53776	How did the audition go?	Comment s'est passÃ©e l'auditionÂ ?
154386	I've no friend to talk to about my problems.	Je n'ai pas d'ami avec lequel je puisse m'entr...
149823	I really like this skirt. Can I try it on?	J'aime beaucoup cette jupe, puis-je l'essayerâ€¯?

In [25]:
```
easy_dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 175621 entries, 2785 to 121958
Data columns (total 2 columns):
 #   Column                  Non-Null Count   Dtype 
---  ------                  --------------   ----- 
 0   English words/sentences  175621 non-null  object
 1   French words/sentences   175621 non-null  object
dtypes: object(2)
memory usage: 4.0+ MB
```

In [26]:
```
easy_dataset["English Words in Sentence"] = (
    easy_dataset["English words/sentences"].str.split().apply(len)
)
easy_dataset["French Words in Sentence"] = (
    easy_dataset["French words/sentences"].str.split().apply(len)
)

fig = px.histogram(
    easy_dataset,
    x=["English Words in Sentence", "French Words in Sentence"],
    color_discrete_sequence=["#3f384a", "#e04c5f"],
    labels={"variable": "Variable", "value": "Words in Sentence"},
    marginal="box",
    barmode="group",
    height=540,
    width=840,
    title="Easy Dataset - Words in Sentence",
)
fig.update_layout(
    font_color=FONT_COLOR,
    title_font_size=18,
    plot_bgcolor=BACKGROUND_COLOR,
    paper_bgcolor=BACKGROUND_COLOR,
    bargap=0.2,
    bargroupgap=0.1,
    legend=dict(orientation="h", yanchor="bottom", xanchor="right", y=1.02, x=1),
    yaxis_title="Count",
)
fig.show()
```

In [27]:
```
sentences_en = easy_dataset["English words/sentences"].to_numpy()
sentences_fr = easy_dataset["French words/sentences"].to_numpy()

valid_fraction = 0.1
valid_len = int(valid_fraction * len(easy_dataset))

sentences_en_train = sentences_en[:-valid_len]
sentences_fr_train = sentences_fr[:-valid_len]

sentences_en_valid = sentences_en[-valid_len:]
sentences_fr_valid = sentences_fr[-valid_len:]
```

In [28]:
```
def prepare_input_and_target(sentences_en, sentences_fr):
    """Return data in the format: `((encoder_input, decoder_input), target)`"""
    # Work with strings only
    return (sentences_en, "startofseq " + sentences_fr), sentences_fr + " endofseq"


def from_sentences_dataset(
    sentences_en,
    sentences_fr,
    batch_size=32,
    cache=True,
    shuffle=False,
    shuffle_buffer_size=10_000,
    seed=None,
):
    """Creates `TensorFlow` dataset for encoder-decoder RNN from given sentences.""
    dataset = tf.data.Dataset.from_tensor_slices((sentences_en, sentences_fr))
    dataset = dataset.map(prepare_input_and_target, num_parallel_calls=tf.data.AUTO
    if cache:
        dataset = dataset.cache()
    if shuffle:
        dataset = dataset.shuffle(shuffle_buffer_size, seed=seed)
    return dataset.batch(batch_size)
```

In [29]:
```
benchmark_ds = from_sentences_dataset(sentences_en_train, sentences_fr_train)
benchmark_ds = benchmark_ds.prefetch(tf.data.AUTOTUNE)
bench_results = tfds.benchmark(benchmark_ds, batch_size=32)
```

```
************ Summary ************

  0%|          | 0/4940 [00:00<?, ?it/s]
Examples/sec (First included) 192589.77 ex/sec (total: 158112 ex, 0.82 sec)
Examples/sec (First only) 1493.88 ex/sec (total: 32 ex, 0.02 sec)
Examples/sec (First excluded) 197709.36 ex/sec (total: 158080 ex, 0.80 sec)
```

In [30]:
```
example_ds = from_sentences_dataset(
    sentences_en_train, sentences_fr_train, batch_size=4
)
list(example_ds.take(1))[0]
```

Out[30]:
```
((<tf.Tensor: shape=(4,), dtype=string, numpy=
  array([b'Take a seat.', b'I wish Tom was here.',
         b'How did the audition go?',
         b"I've no friend to talk to about my problems."], dtype=object)>,
  <tf.Tensor: shape=(4,), dtype=string, numpy=
  array([b'startofseq Prends place !',
         b"startofseq J'aimerais que Tom soit l\xc3\x83\xc2\xa0.",
         b"startofseq Comment s'est pass\xc3\x83\xc2\xa9e l'audition\xc3\x82\xc2\xa0?",
         b"startofseq Je n'ai pas d'ami avec lequel je puisse m'entretenir de mes probl\xc3\x83\xc2\xa8mes."],
        dtype=object)>),
 <tf.Tensor: shape=(4,), dtype=string, numpy=
 array([b'Prends place ! endofseq',
        b"J'aimerais que Tom soit l\xc3\x83\xc2\xa0. endofseq",
        b"Comment s'est pass\xc3\x83\xc2\xa9e l'audition\xc3\x82\xc2\xa0? endofseq",
        b"Je n'ai pas d'ami avec lequel je puisse m'entretenir de mes probl\xc3\x83\xc2\xa8mes. endofseq"],
       dtype=object)>)
```

In [31]:
```
example_ds.cardinality()  # Number of batches per epoch.
```

Out[31]:
```
<tf.Tensor: shape=(), dtype=int64, numpy=39515>
```

In [32]:
```
class ColoramaVerbose(keras.callbacks.Callback):
    def on_epoch_end(self, epoch, logs=None):
        print(
            f"{CLR}Epoch: {RED}{epoch + 1:02d}{CLR} -",
            f"{CLR}loss: {RED}{logs['loss']:.5f}{CLR} -",
            f"{CLR}accuracy: {RED}{logs['accuracy']:.5f}{CLR} -",
            f"{CLR}val_loss: {RED}{logs['val_loss']:.5f}{CLR} -",
            f"{CLR}val_accuracy: {RED}{logs['val_accuracy']:.5f}",
        )
```

In [33]:
```
def adapt_compile_and_fit(
    model,
    train_dataset,
    valid_dataset,
    n_epochs=25,
    n_patience=5,
    init_lr=0.001,
    lr_decay_rate=0.1,
    colorama_verbose=False,
):
    """Takes the model vectorization layers and adapts them to the training data.
    Then, it prepares the final datasets vectorizing targets and prefetching,
    and finally trains the given model. Additionally, provides learning rate schedu
    (exponential decay), early stopping and colorama verbose."""

    model.vectorization_en.adapt(
        train_dataset.map(
            lambda sentences, target: sentences[0],  # English sentences.
            num_parallel_calls=tf.data.AUTOTUNE,
        )
```

```
    )
    model.vectorization_fr.adapt(
        train_dataset.map(
            lambda sentences, target: sentences[1] + b" endofseq",  # French senten
            num_parallel_calls=tf.data.AUTOTUNE,
        )
    )

    train_dataset_prepared = train_dataset.map(
        lambda sentences, target: (sentences, model.vectorization_fr(target)),
        num_parallel_calls=tf.data.AUTOTUNE,
    ).prefetch(tf.data.AUTOTUNE)

    valid_dataset_prepared = valid_dataset.map(
        lambda sentences, target: (sentences, model.vectorization_fr(target)),
        num_parallel_calls=tf.data.AUTOTUNE,
    ).prefetch(tf.data.AUTOTUNE)

    early_stopping_cb = keras.callbacks.EarlyStopping(
        monitor="val_accuracy", patience=n_patience, restore_best_weights=True
    )

    # The line below doesn't work with multi-file interleaving.
    # n_decay_steps = n_epochs * train_dataset_prepared.cardinality().numpy()
    # Less elegant solution.
    n_decay_steps = n_epochs * len(list(train_dataset_prepared))
    scheduled_lr = keras.optimizers.schedules.ExponentialDecay(
        initial_learning_rate=init_lr,
        decay_steps=n_decay_steps,
        decay_rate=lr_decay_rate,
    )

    model_callbacks = [early_stopping_cb]
    verbose_level = 1
    if colorama_verbose:
        model_callbacks.append(ColoramaVerbose())
        verbose_level = 0

    model.compile(
        loss="sparse_categorical_crossentropy",
        optimizer=keras.optimizers.RMSprop(learning_rate=scheduled_lr),
        metrics=["accuracy"],
    )

    return model.fit(
        train_dataset_prepared,
        epochs=n_epochs,
        validation_data=valid_dataset_prepared,
        callbacks=model_callbacks,
        verbose=verbose_level,
    )
```

In [34]:

```
def translate(model, sentence_en):
    translation = ""
    for word_idx in range(model.max_sentence_len):
        X_encoder = np.array([sentence_en], dtype=object)
```

```
        X_decoder = np.array(["startofseq " + translation], dtype=object)
        # Last token's probas.
        y_proba = model.predict((X_encoder, X_decoder), verbose=0)[0, word_idx]
        predicted_word_id = np.argmax(y_proba)
        predicted_word = model.vectorization_fr.get_vocabulary()[predicted_word_id]
        if predicted_word == "endofseq":
            break
        translation += " " + predicted_word
    return translation.strip()
```

In [35]:

```
class BidirectionalEncoderDecoderWithAttention(keras.Model):
    def __init__(
        self,
        vocabulary_size=5000,
        max_sentence_len=50,
        embedding_size=256,
        n_units_lstm=512,
        **kwargs,
    ):
        super().__init__(**kwargs)
        self.max_sentence_len = max_sentence_len

        self.vectorization_en = layers.TextVectorization(
            vocabulary_size, output_sequence_length=max_sentence_len
        )
        self.vectorization_fr = layers.TextVectorization(
            vocabulary_size, output_sequence_length=max_sentence_len
        )

        self.encoder_embedding = layers.Embedding(
            vocabulary_size, embedding_size, mask_zero=True
        )
        self.decoder_embedding = layers.Embedding(
            vocabulary_size, embedding_size, mask_zero=True
        )

        self.encoder = layers.Bidirectional(
            layers.LSTM(n_units_lstm // 2, return_sequences=True, return_state=True
        )
        self.decoder = layers.LSTM(n_units_lstm, return_sequences=True)
        self.attention = layers.Attention()
        self.output_layer = layers.Dense(vocabulary_size, activation="softmax")

    def call(self, inputs):
        encoder_inputs, decoder_inputs = inputs

        encoder_input_ids = self.vectorization_en(encoder_inputs)
        decoder_input_ids = self.vectorization_fr(decoder_inputs)

        encoder_embeddings = self.encoder_embedding(encoder_input_ids)
        decoder_embeddings = self.decoder_embedding(decoder_input_ids)

        # The final hidden state of the encoder, representing the entire
        # input sequence, is used to initialize the decoder.
        encoder_output, *encoder_state = self.encoder(encoder_embeddings)
        encoder_state = [
```

```
            tf.concat(encoder_state[0::2], axis=-1),  # Short-term state (0 & 2).
            tf.concat(encoder_state[1::2], axis=-1),  # Long-term state (1 & 3).
        ]
        decoder_output = self.decoder(decoder_embeddings, initial_state=encoder_sta
        attention_output = self.attention([decoder_output, encoder_output])

        return self.output_layer(attention_output)
```

In [15]:
```
K.clear_session()  # Resets all state generated by Keras.
tf.random.set_seed(42)  # Ensure reproducibility on CPU.

easy_train_ds = from_sentences_dataset(
    sentences_en_train, sentences_fr_train, shuffle=True, seed=42
)
easy_valid_ds = from_sentences_dataset(sentences_en_valid, sentences_fr_valid)

bidirect_encoder_decoder = BidirectionalEncoderDecoderWithAttention(max_sentence_le
bidirect_history = adapt_compile_and_fit(
    bidirect_encoder_decoder,
    easy_train_ds,
    easy_valid_ds,
    init_lr=0.01,
    lr_decay_rate=0.01,
    colorama_verbose=True,
)
```

```
WARNING:tensorflow:From c:\Users\BaddD\miniconda3\Lib\site-packages\keras\src\backend\common\global_state.py:82: The name tf.reset_default_graph is deprecated. Please use tf.compat.v1.reset_default_graph instead.

Epoch: 01 - loss: 2.62662 - accuracy: 0.50105 - val_loss: 1.92284 - val_accuracy: 0.59191
Epoch: 02 - loss: 1.69728 - accuracy: 0.62366 - val_loss: 1.65250 - val_accuracy: 0.63490
Epoch: 03 - loss: 1.43992 - accuracy: 0.66536 - val_loss: 1.52810 - val_accuracy: 0.65608
Epoch: 04 - loss: 1.26064 - accuracy: 0.69638 - val_loss: 1.45221 - val_accuracy: 0.67404
Epoch: 05 - loss: 1.11256 - accuracy: 0.72278 - val_loss: 1.40407 - val_accuracy: 0.68427
Epoch: 06 - loss: 0.98191 - accuracy: 0.74716 - val_loss: 1.36898 - val_accuracy: 0.69315
Epoch: 07 - loss: 0.86706 - accuracy: 0.77014 - val_loss: 1.35017 - val_accuracy: 0.70041
Epoch: 08 - loss: 0.76643 - accuracy: 0.79194 - val_loss: 1.34719 - val_accuracy: 0.70507
Epoch: 09 - loss: 0.67920 - accuracy: 0.81229 - val_loss: 1.35664 - val_accuracy: 0.70751
Epoch: 10 - loss: 0.60512 - accuracy: 0.82975 - val_loss: 1.36325 - val_accuracy: 0.70855
Epoch: 11 - loss: 0.54260 - accuracy: 0.84565 - val_loss: 1.38367 - val_accuracy: 0.70949
Epoch: 12 - loss: 0.49184 - accuracy: 0.85898 - val_loss: 1.39837 - val_accuracy: 0.70935
Epoch: 13 - loss: 0.45000 - accuracy: 0.87069 - val_loss: 1.41815 - val_accuracy: 0.71095
Epoch: 14 - loss: 0.41605 - accuracy: 0.88005 - val_loss: 1.44007 - val_accuracy: 0.70955
Epoch: 15 - loss: 0.38908 - accuracy: 0.88737 - val_loss: 1.45732 - val_accuracy: 0.70900
Epoch: 16 - loss: 0.36676 - accuracy: 0.89380 - val_loss: 1.47583 - val_accuracy: 0.70789
Epoch: 17 - loss: 0.34900 - accuracy: 0.89890 - val_loss: 1.49026 - val_accuracy: 0.70740
Epoch: 18 - loss: 0.33464 - accuracy: 0.90298 - val_loss: 1.50518 - val_accuracy: 0.70633
```

In [16]:
```
fig = px.line(
    bidirect_history.history,
    markers=True,
    height=540,
    width=840,
    symbol="variable",
    labels={"variable": "Variable", "value": "Value", "index": "Epoch"},
    title="Easy Dataset - Encoder-Decoder RNN Training Process",
    color_discrete_sequence=px.colors.diverging.balance_r,
)
fig.update_layout(
    font_color=FONT_COLOR,
    title_font_size=18,
    plot_bgcolor=BACKGROUND_COLOR,
    paper_bgcolor=BACKGROUND_COLOR,
```

```
)
fig.show()
```

In [17]:
```
translation1 = translate(bidirect_encoder_decoder, "Take a seat")
translation2 = translate(bidirect_encoder_decoder, "I wish Tom was here.")
translation3 = translate(bidirect_encoder_decoder, "She ordered him to do it.")

print(CLR + "Actual Possible Translations:")
print(BLUE + "Take a seat".ljust(25), RED + "-> ", BLUE + "Prends place !")
print(
    BLUE + "I wish Tom was here.".ljust(25),
    RED + "-> ",
    BLUE + "J'aimerais que Tom soit là.",
)
print(
    BLUE + "She ordered him to do it.".ljust(25),
    RED + "-> ",
    BLUE + "Elle lui a ordonné de le faire.",
)
print()
print(CLR + "Model Translations:")
print(BLUE + "Take a seat".ljust(25), RED + "-> ", BLUE + translation1)
print(BLUE + "I wish Tom was here.".ljust(25), RED + "-> ", BLUE + translation2)
print(BLUE + "She ordered him to do it.".ljust(25), RED + "-> ", BLUE + translation
```

```
Actual Possible Translations:
Take a seat               ->  Prends place !
I wish Tom was here.      ->  J'aimerais que Tom soit là.
She ordered him to do it. ->  Elle lui a ordonné de le faire.

Model Translations:
Take a seat               ->  assiedstoi
I wish Tom was here.      ->  jaimerais que tom Ã@tait lÃ
She ordered him to do it. ->  elle lui [UNK] de le faire
```

In [36]:
```
class PositionalEncoding(layers.Layer):
    def __init__(
        self, max_sentence_len=50, embedding_size=256, dtype=tf.float32, **kwargs
    ):
        super().__init__(dtype=dtype, **kwargs)
        if not embedding_size % 2 == 0:
            raise ValueError("The `embedding_size` must be even.")

        p, i = np.meshgrid(np.arange(max_sentence_len), np.arange(embedding_size //
        pos_emb = np.empty((1, max_sentence_len, embedding_size))
        pos_emb[:, :, 0::2] = np.sin(p / 10_000 ** (2 * i / embedding_size)).T
        pos_emb[:, :, 1::2] = np.cos(p / 10_000 ** (2 * i / embedding_size)).T
        self.positional_embedding = tf.constant(pos_emb.astype(self.dtype))
        self.supports_masking = True

    def call(self, inputs):
        batch_max_length = tf.shape(inputs)[1]
        return inputs + self.positional_embedding[:, :batch_max_length]
```

In [37]:
```
class Encoder(layers.Layer):
    def __init__(
```

```
        self,
        embedding_size=256,
        n_attention_heads=8,
        n_units_dense=256,
        dropout_rate=0.2,
        **kwargs
    ):
        super().__init__(**kwargs)
        self.multi_head_attention = layers.MultiHeadAttention(
            n_attention_heads, embedding_size, dropout=dropout_rate
        )
        self.feed_forward = keras.Sequential(
            [
                layers.Dense(
                    n_units_dense, activation="relu", kernel_initializer="he_normal
                ),
                layers.Dense(embedding_size, kernel_initializer="he_normal"),
                layers.Dropout(dropout_rate),
            ]
        )
        self.add = layers.Add()
        self.normalization = layers.LayerNormalization()

    def call(self, inputs, mask=None):
        Z = inputs
        skip_Z = Z
        Z = self.multi_head_attention(Z, value=Z, attention_mask=mask)
        Z = self.normalization(self.add([Z, skip_Z]))
        skip_Z = Z
        Z = self.feed_forward(Z)
        return self.normalization(self.add([Z, skip_Z]))


class Decoder(layers.Layer):
    def __init__(
        self,
        embedding_size=256,
        n_attention_heads=8,
        n_units_dense=256,
        dropout_rate=0.2,
        **kwargs
    ):
        super().__init__(**kwargs)
        self.masked_multi_head_attention = layers.MultiHeadAttention(
            n_attention_heads, embedding_size, dropout=dropout_rate
        )
        self.multi_head_attention = layers.MultiHeadAttention(
            n_attention_heads, embedding_size, dropout=dropout_rate
        )
        self.feed_forward = keras.Sequential(
            [
                layers.Dense(
                    n_units_dense, activation="relu", kernel_initializer="he_normal
                ),
                layers.Dense(embedding_size, kernel_initializer="he_normal"),
                layers.Dropout(dropout_rate),
```

```
            ]
        )
        self.add = layers.Add()
        self.normalization = layers.LayerNormalization()

    def call(self, inputs, mask=None):
        decoder_mask, encoder_mask = mask  # type: ignore
        Z, encoder_output = inputs
        Z_skip = Z
        Z = self.masked_multi_head_attention(Z, value=Z, attention_mask=decoder_mas
        Z = self.normalization(self.add([Z, Z_skip]))
        Z_skip = Z
        Z = self.multi_head_attention(
            Z, value=encoder_output, attention_mask=encoder_mask
        )
        Z = self.normalization(self.add([Z, Z_skip]))
        Z_skip = Z
        Z = self.feed_forward(Z)
        return self.normalization(self.add([Z, Z_skip]))
```

In [38]:

```
class Transformer(keras.Model):
    def __init__(
        self,
        vocabulary_size=5000,
        max_sentence_len=50,
        embedding_size=256,
        n_encoder_decoder_blocks=1,
        n_attention_heads=8,
        n_units_dense=256,
        dropout_rate=0.2,
        **kwargs,
    ):
        super().__init__(**kwargs)
        self.max_sentence_len = max_sentence_len

        self.vectorization_en = layers.TextVectorization(
            vocabulary_size, output_sequence_length=max_sentence_len
        )
        self.vectorization_fr = layers.TextVectorization(
            vocabulary_size, output_sequence_length=max_sentence_len
        )
        self.encoder_embedding = layers.Embedding(
            vocabulary_size, embedding_size, mask_zero=True
        )
        self.decoder_embedding = layers.Embedding(
            vocabulary_size, embedding_size, mask_zero=True
        )
        self.positional_encoding = PositionalEncoding(max_sentence_len, embedding_s
        self.encoder_blocks = [
            Encoder(embedding_size, n_attention_heads, n_units_dense, dropout_rate)
            for _ in range(n_encoder_decoder_blocks)
        ]
        self.decoder_blocks = [
            Decoder(embedding_size, n_attention_heads, n_units_dense, dropout_rate)
            for _ in range(n_encoder_decoder_blocks)
        ]
```

```
        self.output_layer = layers.Dense(vocabulary_size, activation="softmax")

    def call(self, inputs):
        encoder_inputs, decoder_inputs = inputs

        encoder_input_ids = self.vectorization_en(encoder_inputs)
        decoder_input_ids = self.vectorization_fr(decoder_inputs)

        encoder_embeddings = self.encoder_embedding(encoder_input_ids)
        decoder_embeddings = self.decoder_embedding(decoder_input_ids)

        encoder_pos_embeddings = self.positional_encoding(encoder_embeddings)
        decoder_pos_embeddings = self.positional_encoding(decoder_embeddings)

        encoder_pad_mask = tf.math.not_equal(encoder_input_ids, 0)[:, tf.newaxis]
        decoder_pad_mask = tf.math.not_equal(decoder_input_ids, 0)[:, tf.newaxis]

        # From original paper: "This masking, combined with fact that the output
        # embeddings are offset by one position, ensures that the predictions for
        # position i can depend only on the known outputs at positions less than i.
        batch_max_len_decoder = tf.shape(decoder_embeddings)[1]
        decoder_causal_mask = tf.linalg.band_part(  # Lower triangular matrix.
            tf.ones((batch_max_len_decoder, batch_max_len_decoder), tf.bool), -1, 0
        )
        decoder_mask = decoder_causal_mask & decoder_pad_mask

        Z = encoder_pos_embeddings
        for encoder_block in self.encoder_blocks:
            Z = encoder_block(Z, mask=encoder_pad_mask)

        encoder_output = Z
        Z = decoder_pos_embeddings
        for decoder_block in self.decoder_blocks:
            Z = decoder_block(
                [Z, encoder_output], mask=[decoder_mask, encoder_pad_mask]
            )

        return self.output_layer(Z)
```

In [23]:

```
K.clear_session()
tf.random.set_seed(42)

transformer = Transformer(max_sentence_len=15)
transformer_history = adapt_compile_and_fit(
    transformer, easy_train_ds, easy_valid_ds, colorama_verbose=True
)
```

```
Epoch: 01 - loss: 2.22650 - accuracy: 0.63253 - val_loss: 1.90051 - val_accuracy:
0.65954
Epoch: 02 - loss: 1.56019 - accuracy: 0.71486 - val_loss: 1.41925 - val_accuracy:
0.73519
Epoch: 03 - loss: 1.25314 - accuracy: 0.76274 - val_loss: 1.17028 - val_accuracy:
0.77464
Epoch: 04 - loss: 1.06217 - accuracy: 0.79349 - val_loss: 1.06233 - val_accuracy:
0.79405
Epoch: 05 - loss: 0.94774 - accuracy: 0.81178 - val_loss: 0.93849 - val_accuracy:
0.81457
Epoch: 06 - loss: 0.87398 - accuracy: 0.82400 - val_loss: 0.88313 - val_accuracy:
0.82332
Epoch: 07 - loss: 0.82452 - accuracy: 0.83207 - val_loss: 0.87083 - val_accuracy:
0.82202
Epoch: 08 - loss: 0.78660 - accuracy: 0.83821 - val_loss: 0.84738 - val_accuracy:
0.83128
Epoch: 09 - loss: 0.75673 - accuracy: 0.84362 - val_loss: 0.80585 - val_accuracy:
0.83620
Epoch: 10 - loss: 0.73152 - accuracy: 0.84794 - val_loss: 0.79323 - val_accuracy:
0.83846
Epoch: 11 - loss: 0.70992 - accuracy: 0.85156 - val_loss: 0.77664 - val_accuracy:
0.84219
Epoch: 12 - loss: 0.69138 - accuracy: 0.85504 - val_loss: 0.76859 - val_accuracy:
0.84230
Epoch: 13 - loss: 0.67521 - accuracy: 0.85782 - val_loss: 0.74208 - val_accuracy:
0.84860
Epoch: 14 - loss: 0.66071 - accuracy: 0.86046 - val_loss: 0.73885 - val_accuracy:
0.84895
Epoch: 15 - loss: 0.64746 - accuracy: 0.86283 - val_loss: 0.72966 - val_accuracy:
0.85136
Epoch: 16 - loss: 0.63581 - accuracy: 0.86536 - val_loss: 0.71969 - val_accuracy:
0.85280
Epoch: 17 - loss: 0.62449 - accuracy: 0.86738 - val_loss: 0.71946 - val_accuracy:
0.85195
Epoch: 18 - loss: 0.61485 - accuracy: 0.86930 - val_loss: 0.70738 - val_accuracy:
0.85546
Epoch: 19 - loss: 0.60507 - accuracy: 0.87108 - val_loss: 0.70499 - val_accuracy:
0.85535
Epoch: 20 - loss: 0.59672 - accuracy: 0.87280 - val_loss: 0.69968 - val_accuracy:
0.85785
Epoch: 21 - loss: 0.58868 - accuracy: 0.87431 - val_loss: 0.69808 - val_accuracy:
0.85690
Epoch: 22 - loss: 0.58105 - accuracy: 0.87577 - val_loss: 0.69465 - val_accuracy:
0.85707
Epoch: 23 - loss: 0.57403 - accuracy: 0.87720 - val_loss: 0.68659 - val_accuracy:
0.85944
Epoch: 24 - loss: 0.56764 - accuracy: 0.87862 - val_loss: 0.68284 - val_accuracy:
0.86040
Epoch: 25 - loss: 0.56148 - accuracy: 0.87971 - val_loss: 0.68275 - val_accuracy:
0.86063
```

In [24]:
```
fig = px.line(
    transformer_history.history,
    markers=True,
    height=540,
    width=840,
```

```
    symbol="variable",
    labels={"variable": "Variable", "value": "Value", "index": "Epoch"},
    title="Easy Dataset - Transformer Training Process",
    color_discrete_sequence=px.colors.diverging.balance_r,
)
fig.update_layout(
    font_color=FONT_COLOR,
    title_font_size=18,
    plot_bgcolor=BACKGROUND_COLOR,
    paper_bgcolor=BACKGROUND_COLOR,
)
fig.show()
```

In [25]:
```
translation1 = translate(transformer, "Take a seat")
translation2 = translate(transformer, "I wish Tom was here.")
translation3 = translate(transformer, "She ordered him to do it.")

print(CLR + "Actual Possible Translations:")
print(BLUE + "Take a seat".ljust(25), RED + "-> ", BLUE + "Prends place !")
print(
    BLUE + "I wish Tom was here.".ljust(25),
    RED + "-> ",
    BLUE + "J'aimerais que Tom soit là.",
)
print(
    BLUE + "She ordered him to do it.".ljust(25),
    RED + "-> ",
    BLUE + "Elle lui a ordonné de le faire.",
)
print()
print(CLR + "Model Translations:")
print(BLUE + "Take a seat".ljust(25), RED + "-> ", BLUE + translation1)
print(BLUE + "I wish Tom was here.".ljust(25), RED + "-> ", BLUE + translation2)
print(BLUE + "She ordered him to do it.".ljust(25), RED + "-> ", BLUE + translation
```

```
Actual Possible Translations:
Take a seat               ->  Prends place !
I wish Tom was here.      ->  J'aimerais que Tom soit là.
She ordered him to do it. ->  Elle lui a ordonné de le faire.

Model Translations:
Take a seat               ->  prenez un siÃ¨ge
I wish Tom was here.      ->  jaimerais que tom soit ici
She ordered him to do it. ->  elle [UNK] de le faire
```

In [4]:
```
chunk_size = 100_000
chunks_dir = Path("data_chunks")

chunks_dir.mkdir(parents=True, exist_ok=True)
chunks = pd.read_csv("dataset/en-fr.csv", chunksize=chunk_size, encoding="utf-8")
print(chunks)
for i, chunk in enumerate(chunks):
    chunk_path = chunks_dir / f"en-fr-chunk-{i:03}.csv"
    chunk.to_csv(chunk_path, index=False, encoding="utf-8")
```

```
<pandas.io.parsers.readers.TextFileReader object at 0x0000016D32E44D70>
```

In [39]:
```
filepaths = [f"{chunks_dir}/{chunk_file}" for chunk_file in os.listdir(chunks_dir)]
filepaths[:10]
```

Out[39]:
```
['data_chunks/en-fr-chunk-000.csv',
 'data_chunks/en-fr-chunk-001.csv',
 'data_chunks/en-fr-chunk-002.csv',
 'data_chunks/en-fr-chunk-003.csv',
 'data_chunks/en-fr-chunk-004.csv',
 'data_chunks/en-fr-chunk-005.csv',
 'data_chunks/en-fr-chunk-006.csv',
 'data_chunks/en-fr-chunk-007.csv',
 'data_chunks/en-fr-chunk-008.csv',
 'data_chunks/en-fr-chunk-009.csv']
```

In [40]:
```
hard_dataset_chunk = pd.read_csv(filepaths[0], encoding="utf-8", engine="pyarrow",
hard_dataset_chunk.head()
```

Out[40]:

	en	fr
0	Changing Lives | Changing Society | How It Wor...	Il a transformé notre vie | Il a transformé la...
1	Site map	Plan du site
2	Feedback	Rétroaction
3	Credits	Crédits
4	Français	English

In [41]:
```
hard_dataset_chunk["English Words in Sentence"] = (
    hard_dataset_chunk["en"].str.split().apply(lambda x: len(x) if x is not None el
)
hard_dataset_chunk["French Words in Sentence"] = (
    hard_dataset_chunk["fr"].str.split().apply(lambda x: len(x) if x is not None el
)

fig = px.histogram(
    hard_dataset_chunk,
    x=["English Words in Sentence", "French Words in Sentence"],
    color_discrete_sequence=["#3f384a", "#e04c5f"],
    labels={"variable": "Variable", "value": "Words in Sentence"},
    marginal="box",
    barmode="group",
    range_x=(-10, 100),
    nbins=500,
    height=540,
    width=840,
    title="Hard Dataset Random Chunk - Words in Sentence",
)
fig.update_layout(
    font_color=FONT_COLOR,
    title_font_size=18,
    plot_bgcolor=BACKGROUND_COLOR,
    paper_bgcolor=BACKGROUND_COLOR,
    bargap=0.2,
```

```
    bargroupgap=0.1,
    legend=dict(orientation="h", yanchor="bottom", xanchor="right", y=1.02, x=1),
    yaxis_title="Count",
)
fig.show()
```

In [42]:
```
def parse_csv_line(line):
    "Decodes `csv` line and returns `(sentence_en, sentence_fr)` tensor."
    defaults = 2 * [tf.constant("", dtype=tf.string)]
    fields = tf.io.decode_csv(line, record_defaults=defaults)
    return tf.stack([fields[0]]), tf.stack([fields[1]])


def prepare_input_and_target(sentences_en, sentences_fr):
    """Return data in the format: `((encoder_input, decoder_input), target)`"""
    # Use TF string ops to safely concatenate start/end tokens with tensors.
    decoder_input = tf.strings.join([tf.constant("startofseq "), sentences_fr])
    target = tf.strings.join([sentences_fr, tf.constant(" endofseq")])
    return (sentences_en, decoder_input), target


def from_csv_files_dataset(
    filepaths,
    batch_size=32,
    cache=True,
    shuffle=False,
    shuffle_buffer_size=50_000,
    seed=None,
):
    """Creates `TensorFlow` dataset from multiple csv files."""
    dataset = tf.data.Dataset.list_files(filepaths, seed=seed)
    dataset = dataset.interleave(  # type: ignore
        lambda filepath: tf.data.TextLineDataset(filepath).skip(1),  # Skip header.
        cycle_length=tf.data.AUTOTUNE,
        num_parallel_calls=tf.data.AUTOTUNE,
    )
    dataset = dataset.map(parse_csv_line, num_parallel_calls=tf.data.AUTOTUNE)
    dataset = dataset.map(prepare_input_and_target, num_parallel_calls=tf.data.AUTO
    if cache:
        dataset = dataset.cache()
    if shuffle:
        dataset = dataset.shuffle(shuffle_buffer_size, seed=seed)
    return dataset.batch(batch_size)
```

In [43]:
```
example_ds = from_csv_files_dataset(filepaths[0:3], batch_size=2)
list(example_ds.take(1))[0]
```

Out[43]:
```
((<tf.Tensor: shape=(2, 1), dtype=string, numpy=
  array([[b'Changing Lives | Changing Society | How It Works | Technology Drives C
hange Home | Concepts | Teachers | Search | Overview | Credits | HHCC Web | Refere
nce | Feedback Virtual Museum of Canada Home Page'],
         [b'July 11-14, 2005, Attended meetings of Inter-Parliamentary Union - Gen
eva']],
        dtype=object)>,
  <tf.Tensor: shape=(2, 1), dtype=string, numpy=
  array([[b'startofseq Il a transform\xc3\xa9 notre vie | Il a transform\xc3\xa9 l
a soci\xc3\xa9t\xc3\xa9 | Son fonctionnement | La technologie, moteur du changemen
t Accueil | Concepts | Enseignants | Recherche | Aper\xc3\xa7u | Collaborateurs |
Web HHCC | Ressources | Commentaires Mus\xc3\xa9e virtuel du Canada'],
         [b"startofseq 11 au 14 juillet 2005, Participe aux r\xc3\xa9unions de l'U
nion interparlementaire - Gen\xc3\xa8ve"]],
        dtype=object)>),
 <tf.Tensor: shape=(2, 1), dtype=string, numpy=
 array([[b'Il a transform\xc3\xa9 notre vie | Il a transform\xc3\xa9 la soci\xc3\x
a9t\xc3\xa9 | Son fonctionnement | La technologie, moteur du changement Accueil |
Concepts | Enseignants | Recherche | Aper\xc3\xa7u | Collaborateurs | Web HHCC | R
essources | Commentaires Mus\xc3\xa9e virtuel du Canada endofseq'],
        [b"11 au 14 juillet 2005, Participe aux r\xc3\xa9unions de l'Union interpa
rlementaire - Gen\xc3\xa8ve endofseq"]],
       dtype=object)>)
```

In [44]:
```
hard_train_ds = from_csv_files_dataset(filepaths[0:2], shuffle=True, seed=42)
hard_valid_ds = from_csv_files_dataset(filepaths[2:3])
```

In [46]:
```
K.clear_session()
tf.random.set_seed(42)

bidirect_encoder_decoder = BidirectionalEncoderDecoderWithAttention()
bidirect_history = adapt_compile_and_fit(
    bidirect_encoder_decoder,
    hard_train_ds,
    hard_valid_ds,
    n_epochs=1,
    colorama_verbose=True,
)
```

Epoch: 01 - loss: 5.23873 - accuracy: 0.18089 - val_loss: 5.09954 - val_accuracy: 0.19158

In [47]:
```
K.clear_session()
tf.random.set_seed(42)

transformer = Transformer()
bidirect_history = adapt_compile_and_fit(
    transformer,
    hard_train_ds,
    hard_valid_ds,
    n_epochs=1,
    colorama_verbose=True,
)
```

Epoch: 01 - loss: 2.32342 - accuracy: 0.61975 - val_loss: 2.17250 - val_accuracy: 0.63486